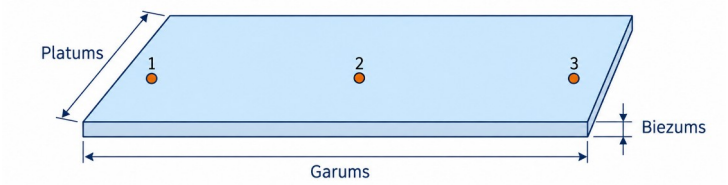
W4: API un datu apstrāde

Ražošanas līnija (p4loksne.py)

Iedomājies, ka strādā plastmasas lokšņu rūpnīcas kvalitātes kontrolē. Klients pasūtījis 1**000 × 500 × 2 mm** loksnes. Sensori katrai loksnei izmēra biezumu trīs vietās, garumu un platumu. Dati ir jāpārveido pārskatā ar kuru operators var novērtēt partijas kvalitāti.



Izmēriem jāiekļaujas dotajās pielaidēs (pieļaujamajos intervālos ap pasūtīto izmēru). Pārāk maza loksne var nederēt klienta detaļai, bet lieks biezums palielina plastmasas patēriņu.

Dati ir simulēti. Pielaides ir dotas šim mācību uzdevumam, nevis pārņemtas no rūpnieciska standarta. Visās partijās analizē vienāda izstrādājuma izmērus. Abas intervāla robežas ir pieļaujamas. Loksne atbilst tikai tad, ja garums, platums un katrs no trim biezuma mērījumiem ir pielaidēs.

Izmērs	**Mērķis**	**Pieļaujamais intervāls**
Garums	1000 mm	999–1001 mm
Platums	500 mm	499–501 mm
Biezums katrā vietā	2 mm	1,95–2,05 mm

Datnē **p4loksne.py** uzrakstīsim programmu, kas pieprasa JSON datnes URL, iegūst datus ar **requests** bibliotēku un aprēķina partijas kvalitātes rādītājus.

Datu struktūras piemērs:

```
[
  {
    "batch_id": "P001",
    "sheet_number": 1,
    "length_mm": 1000.25,
    "width_mm": 499.90,
    "thickness_mm": [1.98, 2.01, 2.03]
  },
  {
    "batch_id": "P001",
    "sheet_number": 2,
    "length_mm": 1000.2,
    "width_mm": 500.0,
    "thickness_mm": [1.99, 2.0, 2.0]
  },
 ...
]
```

Atslēga	Nozīme
batch_id	Partijas identifikators
sheet_number	Loksnes kārtas numurs partijā
length_mm	Garums milimetros
width_mm	Platums milimetros
thickness_mm	Trīs biezumi: kreisā mala, vidus, labā mala

Partijās ir atšķirīgs lokšņu skaits. Datnes nosaukums ir norāde par simulēto situāciju – kvalitātes rezultāts jāaprēķina no mērījumiem.

Datu iegūšana

Nosūta HTTP pieprasījumu un pārvērš atbildi Python datu struktūrā.

```
import requests #importē bibliotēku

# adrese, kur ir dati
url = "https://github.com/okskola/matinz/raw/refs/heads/main/W4/P002_biezums_aug.json"

try:
    response = requests.get(url, timeout=10) # iegūst datus no tīmekļa
    response.raise_for_status() # kļūdas gadījumā izraisa izņēmumsituāciju
    data = response.json() # pārveido JSON par Python datu struktūrām
except requests.exceptions.JSONDecodeError: # izņēmumsituācija - nav atgriezts JSON
    print("Atbilde nav derīgs JSON.")
    exit()
except requests.RequestException: # izņēmumsituācija, ja serveris neatgrieza datus
    print("Neizdevās iegūt datus. Pārbaudi adresi un savienojumu.")
    exit()

# Šeit turpini datu apstrādi.
print("Dati ir iegūti")
```

Iegūto paraugu skaits

Dotajā JSON struktūrā vispirms ir saraksts, kas satur katra parauga datus kā vārdnīcu. Lai noskaidrotu saraksta elementu skaitu, lieto funkciju len().

```
print("Lokšņu skaits:", len(data))
```

Viena parauga pārbaude

Izveido funkciju is_acceptable(sheet), kas atgriež True vai False. Sāc ar garumu un platumu, pēc tam ar ciklu pārbaudi visus biezumus. Vidējais biezums nav atbilstības kritērijs.

```
def is_acceptable(sheet):
    if sheet["length_mm"] < 999 or sheet["length_mm"] > 1001:
        return False

    if sheet["width_mm"] < 499 or sheet["width_mm"] > 501:
        return False

    for thickness in sheet["thickness_mm"]:
        if thickness < 1.95 or thickness > 2.05:
            return False
    return True
```

Uzdevums – lokšņu partijas pārskats

Apstrādā visas loksnes un terminālī izvada:

- partijas identifikatoru un kopējo lokšņu skaitu;
- atbilstošo un neatbilstošo lokšņu skaitu;
- neatbilstošo lokšņu procentu ar vienu zīmi aiz komata;
- neatbilstošo lokšņu numurus.

Vienu loksni kopējā brāķa skaitā ieskaita tikai vienreiz.

Pārbaude – izvada visas loksnes

Šis cikls apstrādā katru loksni un paziņo vai tā ir derīga.

```
for sheet in data:
    if is_acceptable(sheet):
        print(f"Loksne {sheet['sheet_number']}: der")
    else:
        print(f"Loksne {sheet['sheet_number']}: neder")
```

Cikls, kas saskaita, cik loksnes atbilst

Veido skaitītāju, kas saskaita derīgās loksnes.

```
acceptable_count = 0
for sheet in data:
    if is_acceptable(sheet):
        acceptable_count = acceptable_count + 1
```

Veido sarakstu ar neatbilstošajām loksnēm

Pirms cikla izveido tukšu sarakstu, kam pievieno nederīgo lokšņu numurus.

```
acceptable_count = 0
unacceptable_numbers = [] # izveido tukšu sarakstu
for sheet in data:
    if is_acceptable(sheet):
        acceptable_count = acceptable_count + 1
    else:
        # pievieno saraksta beigās vienu elementu
        unacceptable_numbers.append(sheet["sheet_number"])
```

Patstāvīgais darbs

Izvadi terminālī uzdevumā prasītos datus. Ņem vērā, ka partijā var nebūt neviens mērījums.

Datnes testēšanai

```
https://github.com/okskola/matinz/raw/refs/heads/main/W4/P001_stabila.json
https://github.com/okskola/matinz/raw/refs/heads/main/W4/P002_biezums_aug.json
https://github.com/okskola/matinz/raw/refs/heads/main/W4/P003_nevienmeriga.json
https://github.com/okskola/matinz/raw/refs/heads/main/W4/P004_garuma_kludas.json
https://github.com/okskola/matinz/raw/refs/heads/main/W4/P005_reguleta.json
https://github.com/okskola/matinz/raw/refs/heads/main/W4/P006_tukss.json
```

Iesūtīšana (check50 testu nav – jātestē patstāvīgi)

```
submit50 okskola/cs50addp/mzin/p4loksne
```

Diagramma (p4diagr.py)

Lietosim tos pašus datus un bibliotēkas **statistics** un **matplotlib**, lai:

- aprēķinātu katras loksnes vidējo biezumu;
- aprēķinātu katras loksnes biezumu atšķirību (biezākais - plānākais);
- atrastu loksni ar lielāko biezuma atšķirību;
- diagrammā attēlotu katras loksnes vidējo biezumu

un līdzīgu programmas sagatavi, kas iegūst šos datus.

Koda sagatave

```
import requests
import statistics
import matplotlib.pyplot as plt

url = "https://github.com/okskola/matinz/raw/refs/heads/main/W4/P002_biezums_aug.json"

try:
    response = requests.get(url, timeout=10)
    response.raise_for_status()
    data = response.json()
except requests.exceptions.JSONDecodeError:
    print("Atbilde nav derīgs JSON.")
    exit()
except requests.RequestException:
    print("Neizdevās iegūt datus. Pārbaudi adresi un savienojumu.")
    exit()

for sheet in data:
    pass
```

Bibliotēka statistics

https://docs.python.org/3/library/statistics.html

Tā ir Python standarta bibliotēka vienkāršiem statistikas aprēķiniem. Tā nav papildus jāinstalē. Dati ir saraksts ar skaitliskām vērtībām. Galvenās funkcijas aprēķina:

- mean(dati) – Aritmētisko vidējo;
- median(dati) – Sakārtotu datu vidējo vērtību; pāra skaita gadījumā — divu vidējo vērtību vidējo;
- mode(dati) – Visbiežāk sastopamo vērtību;
- stdev(dati) – Izlases standartnovirzi (datu izkliedes rādītāju);
- variance(dati) – Izlases dispersiju (standartnovirzes kvadrātu).

Piemēram:

```
statistics.mean([1.98, 2.00, 2.02])  # 2.0
```

Vidējo biezumu aprēķins

Mūsu datos ir tikai trīs mērījumi, tādēļ vidēju varētu aprēķināt arī kā (b1+b2+b3)/3. Rakstot programmu labāk ir pieņemt, ka kādreiz iekārta var mainīties un, piemēram, veikt daudz vairāk biezuma mērījumus. Tad ir divas iespējas – rakstīt pašiem ciklu, kas aprēķina vidējo biezumu vai arī lietot bibliotēkas funkciju, kas to dara. Bieži vien bibliotēka ir rakstīta citā programmēšanas valodā un strādā daudz ātrāk par Python ciklu (Python ir salīdzinoši lēni strādājoša valoda).

Saglabāsim katras loksnes vidējo biezumu sarakstā – tas noderēs vēlāk, lai zīmētu diagrammu.

```
averages = [] # izveido tukšu sarakstu

for sheet in data:
    avg = statistics.mean(sheet["thickness_mm"]) # aprēķina vidējo biezumu
    averages.append(avg) # pievieno to sarakstam
```

Bibliotēka matplotlib

https://matplotlib.org/stable/index.html

Tā ir Python bibliotēka diagrammu veidošanai, kas nav iekļauta Python komplektā, bet ir jāinstalē atsevišķi. Arī requests bibliotēka ir jāinstalē atsevišķi, bet CS50 vidē abas jau ir instalētas. Ja datoram ir grafiskā vide, tad diagrammu var attēlot jaunā logā. CS50 vidē grafiskā vide ir, bet tās lietošana nav sevišķi ērta, tādēļ vienkāršāk ir ģenerēt attēlu, kas tiek saglabāts mapē.

Bibliotēkai ir ļoti daudz iespēju. Dažas biežāk lietotās funkcijas:

- plot(x, y) – Zīmē līniju diagrammu;
- bar(x, y) – Zīmē stabiņu diagrammu;
- plt.pie() – veido sektoru diagrammu;
- scatter(x, y) – zīmē punktu diagrammu (katrs punkts attēlo vienu X un Y vērtību pāri);
- xlabel(teksts) – Nosauc X asi;
- ylabel(teksts) – Nosauc Y asi;
- title(teksts) – Pievieno virsrakstu;
- savefig(datne) – Saglabā diagrammu attēlā.

Līniju un stabiņu diagrammai nodod divus sarakstus (x un y ass datus), piemēram:

```
plt.bar(["Der", "Neder"], [80, 20])
plt.ylabel("Lokšņu skaits")
plt.title("Partijas kvalitāte")
plt.savefig("diagramma1.png")
```

Piemērs sektoru diagrammai:

```
plt.pie([80, 20], labels=["Der", "Neder"], autopct="%1.1f%%")
plt.title("Partijas kvalitāte")
plt.savefig("diagramma2.png")
```

Vidējo biezumu diagramma

Jau ir aprēķināti vidējie biezumi (y ass dati), bet diagrammai vajag arī x ass datus, tādēļ izveidosim vēl vienu sarakstu ar detaļu numuriem.numbers = []

```
averages = [] # izveido tukšu sarakstu vidējām vērtībām
numbers = [] # izveido tukšu sarakstu lokšņu numuriem
for sheet in data:
    numbers.append(sheet["sheet_number"])
    avg = statistics.mean(sheet["thickness_mm"]) # aprēķina vidējo biezumu
    averages.append(avg) # pievieno to sarakstam
```

Un uzzīmēsim diagrammu:

```
plt.plot(numbers, averages, marker=".", linestyle="None")
plt.xlabel("Loksnes numurs")
plt.ylabel("Vidējais biezums (mm)")
plt.savefig("videjais_biezums.png")
```

Patstāvīgais darbs

1. Ar statistics.mean() aprēķini partijas vidējo garumu un platumu. Lai to izdarītu vispirms ciklā ir jāizveido atsevišķs garumu saraksts un atsevišķs platumu saraksts.

2. Izvadi vidējā garuma un platuma novirzi no normas (garums 1000 mm un platums 500mm).

3. Izveido stabiņu diagrammu, kurā parādītas abas novirzes. Ja vienkārši tiks izsaukta jauna plt funkcija, tad tā papildinās iepriekšējo diagrammu. Tāpēc ir jālieto “figūras”. Pirms katras jaunas diagrammas izsauc funkciju plt.figure(), bet pēc diagrammas plt.close(). Piemēram:

```
plt.figure()
plt.scatter(numbers, averages)
plt.xlabel("Loksnes numurs")
plt.ylabel("Vidējais biezums (mm)")
plt.savefig("videjais_biezums.png")
plt.close()
```

4. Izmanto iepriekšējā uzdevuma funkciju un izveido sektora diagrammu ar atbilstošo un neatbilstošo lokšņu attiecību.

Iesūtīšana (check50 testu nav – jātestē patstāvīgi)

```
submit50 okskola/cs50addp/mzin/p4diagr
```

Datu kļūdas (p4kludas.py)

Izmēģini iepriekšējo programmu ar šiem datiem:

- https://github.com/okskola/matinz/raw/refs/heads/main/W4/P007_keyerror.json
- https://github.com/okskola/matinz/raw/refs/heads/main/W4/P008_typeerror.json
- https://github.com/okskola/matinz/raw/refs/heads/main/W4/P009_statistics_error.json

Kļūdas! Nepareizi dati var apturēt programmu, jo notiek izņēmumsituācijas. Tās ir jāapstrādā!

Biežākās problēmas apstrādājot datus:

Vārdnīcā trūkst prasītās atslēgas	KeyError
Nepiemērots datu tips, piemēram, tekstu salīdzina ar skaitli	TypeError
Vērtību nevar pārveidot, piemēram, float("abc")	ValueError
Piekļūst neesošam saraksta elementam	IndexError
Aprēķinā dala ar nulli	ZeroDivisionError
statistics.mean() saņem tukšu sarakstu	statistics.StatisticsError
Saņemtajam JSON ir nederīga sintakse	requests.exceptions.JSONDecodeError

Patstāvīgais darbs

Izveido iepriekšējās programmas kopiju ar nosaukumu **p4kludas.py** un pievieno kļūdu apstrādi (try/except, norādot konkrēto kļūdas tipu(s), ko apstrādāsi). Svarīgi – viena nepareiza loksne nedrīkst apturēt citu lokšņu datu apstrādi, tāpēc try/except raksti cikla iekšienē, nevis ārpus cikla. Svarīgi – ja nevarēja apstrādāt jebkuru no loksnes datiem, tad visu loksni (visus datus par loksni) nedrīkst pievienot sarakstam. Citādi sanāks, ka, piemēram, loksnes numurs ir pievienots, bet garums nē – tas nojauks aprēķinus un diagrammu veidošanu. Ieteikums – vispirms datus no vārdnīcas ieraksti pagaidu mainīgajos un šo daļu ievieto izņēmumsituāciju tveršanas try/except blokā. Ja kļūdu nebija, tad datus var pievienot sarakstiem, piemēram:

```
errors = 0
v1_list = []
v2_list = []
for element in data:
    try:
        value1 = element["value1"]
        value2 = element["value2"]
    except KeyError:
        errors = errors + 1
    else:
        v1_list.append( value1 )
        v2_list.append( value2 )
```

Kļūdu gadījumā izvadi kļūdaino lokšņu skaitu, kas netika ņemti vērā aprēķinos!

Iesūtīšana (check50 testu nav – jātestē patstāvīgi)

```
submit50 okskola/cs50addp/mzin/p4kludas
```